INFO0947 : Compléments de Programmation
Tableau et Zigzag

B. Donnet, G. Brieven
Université de Liège

Alerte : Évitez de vous retrouver bêtement dans une situation inconfortable ...

- Les projets sont à réaliser seuls, en binôme !
- Les projets doivent être réalisés par vous-même !
- Votre solution ne pourra pas être générée entièrement ou partiellement à l'aide d'un outil d'Intelligence Artificielle (e.g., ChatGPT, Blackbox, etc.).

1 Problème

On considère un tableau d'entiers T de taille N (avec $N \geq 1$). Un sous-tableau contigu est appelé *zigzag* si ses éléments alternent strictement entre montée et descente.

Autrement dit, un segment

$$T[i], T[i+1], \ldots, T[j]$$

est zigzag s'il vérifie l'une des deux conditions suivantes :

$$T[i] < T[i+1] > T[i+2] < T[i+3] > \cdots$$

ou

$$T[i] > T[i+1] < T[i+2] > T[i+3] < \cdots$$

Un segment zigzag doit avoir une longueur d'au moins 2. Le seul cas où un tableau ne contient aucun zigzag est donc lorsque tous ses éléments sont identiques.

L'objectif de cet exercice est de déterminer, dans le tableau donné, le plus long segment zigzag et d'en extraire les informations suivantes :

- la **position de début** du segment,
- la **position de fin**,
- sa **longueur**,
- la **somme des éléments** qui le composent.

En cas d'égalité de longueur entre plusieurs segments zigzag maximaux, on retiendra le **premier**.

Et dans le cas où le tableau ne contient aucun zigzag, les positions de début et de fin sont égales à 0, la longueur doit valoir 1 et la somme des éléments correspond à la valeur de la première case (identique à toutes les autres).

Ainsi, par exemple, pour le tableau suivant :

$$T = \{2, 2, 3, 1, 4, 6, 0, -1\}$$

Les segments zigzag sont par exemple :

$$\{2, 3, 1, 4\} \quad \text{(longueur 4)}$$

$$\{4, 6, 0\} \quad \text{(longueur 3)}$$

Le plus long segment zigzag est donc :

$$\{2, 3, 1, 4\}$$

La fonction devra retourner :
— `start = 1`,
— `end = 4`,
— `length = 4`,
— `sum = 10`.
Le fichier `main-zigzag.c` vous donne plusieurs exemples de zigzags dans différents tableaux.

Dans ce travail, nous vous demandons de construire la fonction suivante :

```
void analyse_zigzag(int *T, unsigned int N, unsigned int *start, unsigned int *end,
                                            unsigned int *length, int *sum);
```

où `T` est un tableau de taille `N`, avec `N` ≥ 1. Cette fonction ne modifie pas `T`. Les quatre derniers paramètres servent à renvoyer les valeurs suivantes :
— `start` : indice du début du segment zigzag maximal,
— `end` : indice de fin du segment,
— `length` : longueur du segment trouvé,
— `sum` : somme des éléments du segment.
Nous vous demandons d'implémenter la fonction `zigzag()` en respectant les contraintes suivantes :
— Aucun tableau intermédiaire n'est autorisé.
— Seules les bibliothèques `assert.h` et `stdlib.h` peuvent être utilisées.
— La solution doit être en complexité temporelle $\mathcal{O}(N)$.
— Le tableau `T` ne doit pas être modifié.
— La solution doit utiliser une et une seule boucle `while`.

2 Travail à Réaliser

On demande, pour le module `zigzag()`, de résoudre le problème en suivant le cheminement suivant :

1. formaliser (= utiliser une notation mathématique concise et précise pour décrire) le problème. À cette fin, proposer de nouvelles notations et/ou de nouveaux concepts ;
2. spécifier le plus précisément et formellement possible le module `zigzag()` ;
3. donner une découpe en Sp respectant les contraintes du problème donné et indiquer comment ils s'emboitent (si applicable). Spécifier complètement et formellement chaque nouveau Sp ;
4. proposer un Invariant Graphique et un Invariant Formel pour chaque boucle de votre code ;
5. construire des morceaux de programme correspondant à chaque Sp (en adoptant l'*approche constructive* vue au cours) ;
6. rédiger le module final ;
7. démontrer de manière rigoureuse et formelle [1] la complexité théorique, dans le pire des cas (i.e., en utilisant la notation de Landau), de votre solution.

Lors de la rédaction du code, veillez à respecter les principes de la **programmation défensive**.

3 Aspects Pratiques

Le travail à rendre se compose d'une archive `tar.gz` [2]. Votre archive portera le nom `zigzag-groupeXX.tar.gz`, où `XX` fait référence à l'identifiant de votre groupe [3].

1. Toute justification impliquant un texte argumentatif en lieu et place d'équations ne sera pas lue. Expliquez tout de même votre raisonnement.
2. Nous vous renvoyons à la formation sur la ligne de commande, donnée au Q1, pour la compression des fichiers dans une archive `tar.gz`.
3. Pour rappel, il est interdit de changer de groupe entre les différents projets. Votre identifiant doit nécessairement être un nombre composé de deux chiffres (compris entre 01 et 50).

Une fois décompressée, votre archive devra donner naissance à deux répertoires : `rapport/` (cfr. Sec. 3.1) et `code/` (Sec. 3.2).

Tout non-respect des consignes se verra sanctionné par 2 points en moins dans la note finale de votre projet [4].

3.1 Rapport

Votre rapport devra être rédigé via l'outil LaTeX(nous vous renvoyons à la formation donnée le CSS, le 05/02/2026, pour la rédaction d'un document en LaTeX) en utilisant le template LaTeXdisponible sur eCampus (Sec. Projets).

Le template est organisé comme suit :

- `main.tex`. C'est le fichier principal. Vous y trouverez une section « TITRE » délimitée par des commentaires. Il suffit de compléter les définitions des macros `\intitule`, `\GrNbr`, `\PrenomUN`, `\NomUN`, `\PrenomDEUX` et `\NomDEUX` par, respectivement, le titre du projet, votre numéro de groupe, vos prénoms et noms. **Attention !** Ne modifiez pas les dix lignes appelées « *Zone protégée* ».
- `introduction.tex`. Dans ce fichier, vous rédigez l'introduction de votre rapport (inutile de recopier l'énoncé du projet, essayez d'y mettre du vôtre).
- `formalisation.tex`. Dans ce fichier, introduisez toutes les notations mathématiques que vous jugez utiles à la réalisation du projet.
- `analyse.tex`. Dans ce fichier, vous devez définir (Input/Output/Caractérisation des Inputs) proprement et clairement le problème. Si applicable, il est aussi demandé de réaliser une analyse complète (i.e., découpe en Sp).
- `specifications.tex`. Dans ce fichier, pécifiez formellement chacun des Sp.
- `invariants.tex`. Dans ce fichier, indiquez et décrivez tous les Invariants de Boucle nécessaires. Pour chaque Sp nécessitant un Invariant de Boucle (une sous-section/Sp), donnez l'Invariant Graphique et l'Invariant Formel correspondant. Pour l'Invariant Graphique, nous vous rappelons qu'il est possible d'exporter les Invariants Graphiques au format PDF depuis le Gli.
- `construction.tex`. Dans ce fichier, il est demandé d'appliquer l'approche constructive pour la construction de votre code ({PréCondition} Init {Inv}, ...).
- `code.tex`. Dans ce fichier, indiquez le code complet (sans assertions intermédiaires) de votre solution.
- `complexite.tex`. Dans ce fichier, vous devez étudier complètement la complexité de votre code. Soyez le plus formel (i.e., mathématique) possible.
- `conclusion.tex`. Dans ce fichier, vous indiquez la conclusion de votre rapport.

Soyez clairs et précis dans vos explications. N'hésitez pas à ajouter un schéma si vous le jugez nécessaire. Dans ce cas, expliquez-le : un schéma seul ne suffit pas !

Soignez votre orthographe : au delà de trois fautes, chaque faute vous fera perdre 0,25 points.

Pour compiler votre rapport, renommez `"main.tex"` en `"zigzag-XX.tex"` et utiliser la commande suivante : `"pdflatex zigzag-XX.tex"`

Une fois compilé, votre document LaTeXdevra produire un fichier PDF dont le nom sera `zigzag-XX.pdf`, où `XX` représente toujours votre numéro de groupe. Ce rapport ne pourra pas compter plus de **12 pages**. [5]

Le dossier `rapport/` sera composé des éléments suivants :

- votre rapport au format `.tex` ainsi que toutes les sources utiles à la compilation ;
- votre rapport déjà compilé au format `.pdf`

Remarque : Pour inclure du code C dans un document LaTeX, voir la remarque publiée sur le forum eCampus (Projets – Informations Générales → Inclure du code dans le rapport + conseils latex).

4. De plus, toute archive ne pouvant être décomposée (c'est-à-dire une archive corrompue) ou ne produisant pas les dossiers `rapport/` et `code/` lors de la décompression engendrera une note finale **nulle**.
5. Sans compter la page de garde, ni son verso.

3.2 Code Source C

Votre code source devra être rédigé en langage C. Votre code source devra être accompagné d'un `Makefile`[6] rédigé par vos soins, la commande `make` permettant la génération d'un fichier binaire exécutable appelé `zigzag`. Ce fichier exécutable sera situé dans le même répertoire que celui où se trouve le code source (c'est-à-dire le répertoire `code/`). Son exécution doit nous permettre de tester la validité de votre code. La fonction `main` de cet exécutable devra être implémentée dans un fichier appelé `main-zigzag.c`.

Votre code source sera adéquatement découpé en headers et modules. Les différents sous-problèmes identifiés dans votre rapport devront apparaitre clairement dans votre code et être déclaré dans le fichier `zigzag.h`. Nous vous fournissons une première version du fichier `zigzag.h` qui, à ce stade, ne contient que la déclaration du module `zigzag()`. Il vous faut donc implémenter le contenu de `zigzag.h` dans un fichier `zigzag.c`.

Le module `zigzag()` (et donc le fichier `zigzag.c`) doit pouvoir être compilé et utilisé sans la présence du fichier `main-zigzag.c`.

Il est **interdit**[7] d'utiliser des fonctions ou procédures de librairies tierces (y compris la librairie standard du C), à l'exception de `assert.h` et `stdlib.h`. De plus, l'instruction `return` ne peut pas apparaitre dans le corps de vos boucles, puique cela va à l'encontre de l'approche constructive. C'est le gardien qui doit gouverner la sortie de la boucle.

4 Agenda

4.1 Milestones

Pendant la durée du projet, nous vous proposons deux *milestones*. L'objectif de ces milestones est de rencontrer chaque groupe individuellement afin de discuter de l'avancement du projet et corriger le tir le cas échéant. Ces rencontres sont organisées durant resp. la 3e et la 2e semaines qui précèdent la remise du travail final. À l'issue de ces rencontres, aucune note ne sera affectée. L'objectif est donc de réaliser une évaluation formative de votre travail pour vous aider à réaliser le projet et à en tirer des enseignements.

Les deux milestones sont les suivants :

- **semaine du 09/03/2026**. Le milestone portera sur les notations/formalisations du problème. Afin de permettre à l'équipe pédagogique de préparer au mieux le feedback, il vous est demandé de nous (i.e., Benoit Donnet et Géraldine Brieven) envoyer par email un petit document[8] (pdf, rédigé en LaTeX) décrivant les différentes notations que vous comptez introduire. La deadline pour l'envoi de ce document est le samedi 07/03/2026, 12h00. Vous devez spécifier, dans l'objet de votre email, la chaîne de caractères suivantes : `[INFO0947] Projet 1 - Milestone 1`
- **semaine du 16/03/2026**. Le milestone portera sur les Invariants de Boucle nécessaires à la réalisation du problème. Afin de permettre à l'équipe pédagogique de préparer au mieux le feedback,il vous est demandé de nous (i.e., Benoit Donnet et Géraldine Brieven) envoyer par email un petit document (pdf, rédigé en LaTeX) décrivant vos Invariants de Boucle (Invariant Graphique et Invariant Formel) que vous comptez introduire. La deadline pour l'envoi de ce document est le samedi 14/03/2026, 12h00. Vous devez spécifier, dans l'objet de votre email, la chaîne de caractères suivantes : `[INFO0947] Projet 1 - Milestone 2`

Vous trouverez, sur eCampus, un template LaTeXpour formater correctement votre production pour chaque milestone.

La participation à ces milestones est obligatoire. Aucun groupe ne peut s'y soustraire. Pour définir le moment auquel votre groupe viendra rencontrer l'équipe pédagogique, deux sujets de discussion ont été créés dans le forum du cours, sur la plateforme eCampus. Il vous suffit de lire et de suivre les instructions détaillées dans le premier message de chaque sujet.

Veuillez faire débuter les sujets des mails envoyés à l'équipe pédagogique par la chaine "**[INFO0947]**".

6. cfr le cours INFO0030, "Partie 2 – Chapitre 1 : Compilation".
7. Cette restriction ne s'applique pas au fichier `main-zigzag.c` où est implémenté l'exécutable de test `zigzag`
8. Petit = 1 page.

4.2 Deadline

Les archives `tar.gz` finales doivent être soumises sur la Plateforme de Soumission avant le **30/03/2026** à **18h00** (l'heure du serveur faisant foi). Toute soumission postérieure à cette date ne sera pas prise en compte. Comme l'heure de l'horloge du serveur de soumission et celle de votre ordinateur peuvent être légèrement différentes, il est **fortement déconseillé** de soumettre votre projet à 17h59 et 59 secondes : ne prenez pas de risque inutile.